WIA1006/WID3006 MACHINE LEARNING

Semester 2, Session 2025/2026 | Universiti Malaya

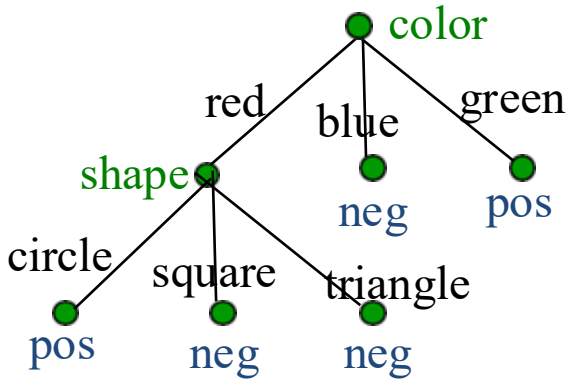
DECISION TREE

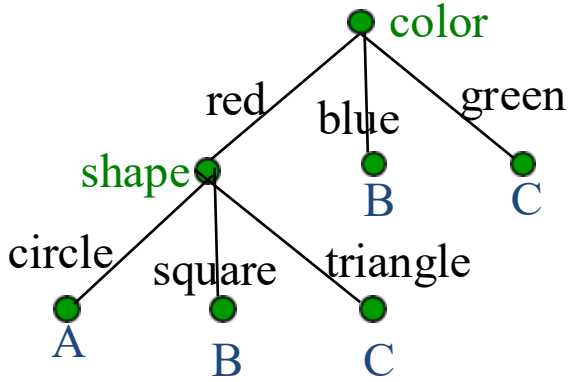
Decision Tree

- Tree-based classifiers for instances represented as feature-vectors. Nodes test features, there is one branch for each value of the feature, and leaves specify the category.
- Can represent arbitrary conjunction and disjunction.
- Can represent any classification function over discrete feature vectors.
- Can be rewritten as a set of rules, i.e. disjunctive normal form (DNF).
  - red ∧ circle → pos
  - red ∧ circle → A

    blue → B; red ∧ square → B

    green → C; red ∧ triangle → C





Properties of Decision Tree Learning

- Continuous (real-valued) features can be handled by allowing nodes to split a real valued feature into two ranges based on a threshold (e.g. length < 3 and length ≥3)
- Classification trees have discrete class labels at the leaves, ***regression trees*** allow real-valued outputs at the leaves.
- Algorithms for finding consistent trees are efficient for processing large amounts of training data for data mining tasks.
- Methods developed for handling noisy training data (both class and feature noise).
- Methods developed for handling missing feature values.

$$(Outlook = Sunny \wedge Humidity = Normal)$$
$$\vee \quad (Outlook = Overcast)$$
$$\vee \quad (Outlook = Rain \wedge Wind = Weak)$$

FIGURE 3.1
A decision tree for the concept *PlayTennis*. An example is classified by sorting it through the tree to the appropriate leaf node, then returning the classification associated with this leaf (in this case, *Yes* or *No*). This tree classifies Saturday mornings according to whether or not they are suitable for playing tennis.

Problems for Decision Tree Learning

- ***Instances are represented by attribute-value pairs.*** Instances are described by a fixed set of attributes (e.g., Temperature) and their values (e.g., Hot). The easiest situation for decision tree learning is when each attribute takes on a small number of disjoint possible values (e.g., Hot, Mild, Cold). However, extensions to the basic algorithm allow handling real-valued attributes as well (e.g., representing Temperature numerically).
- ***The target function has discrete output values.*** The decision tree in Figure 3.1 assigns a boolean classification (e.g., yes or no) to each example. Decision tree methods easily extend to learning functions with more than two possible output values. A more substantial extension allows learning target functions with real- valued outputs, though the application of decision trees in this setting is less common.
- ***Disjunctive descriptions may be required.*** As noted above, decision trees naturally represent disjunctive expressions.

Problems for Decision Tree Learning

- ***The training data may contain errors.*** Decision tree learning methods are robust to errors, both errors in classifications of the training examples and errors in the attribute values that describe these examples.
- ***The training data may contain missing attribute values***. Decision tree methods can be used even when some training examples have unknown values (e.g., if the Humidity of the day is known for only some of the training examples).

- **CLASSIFICATION PROBLEMS**

Decision Tree Induction Pseudocode

DTree(*examples*, *features*) returns a tree
  If all *examples* are in one category, return a leaf node with that category label.
  Else if the set of *features* is empty, return a leaf node with the category label that
    is the most common in examples.
  Else pick a feature F and create a node R for it
    For each possible value v_i of F:
      Let $examples_i$ be the subset of examples that have value v_i for F
      Add an out-going edge E to node R labeled with the value v_i.
      If $examples_i$ is empty
        then attach a leaf node to edge E labeled with the category that
          is the most common in *examples*.
        else call DTree($examples_i$, $features - \{F\}$) and attach the resulting
          tree as the subtree under edge E.
    Return the subtree rooted at R.

Picking a Good Split Feature

- Goal is to have the resulting tree be as small as possible, per Occam's razor.
- Finding a minimal decision tree (nodes, leaves, or depth) is an NP-hard optimization problem.
- Top-down divide-and-conquer method does a greedy search for a simple tree but does not guarantee to find the smallest.
  - General lesson in ML: "Greed is good."
- Want to pick a feature that creates subsets of examples that are relatively "pure" in a single class so they are "closer" to being leaf nodes.
- There are a variety of heuristics for picking a good test, a popular one is based on ***information gain*** that originated with the ID3 system of Quinlan (1979).

Which attribute is the best classifier?

- ***Information gain -*** that measures how well a given attribute separates the training examples according to their target classification
- ***Entropy*** - that characterizes the (im)/purity of an arbitrary collection of examples.

Entropy

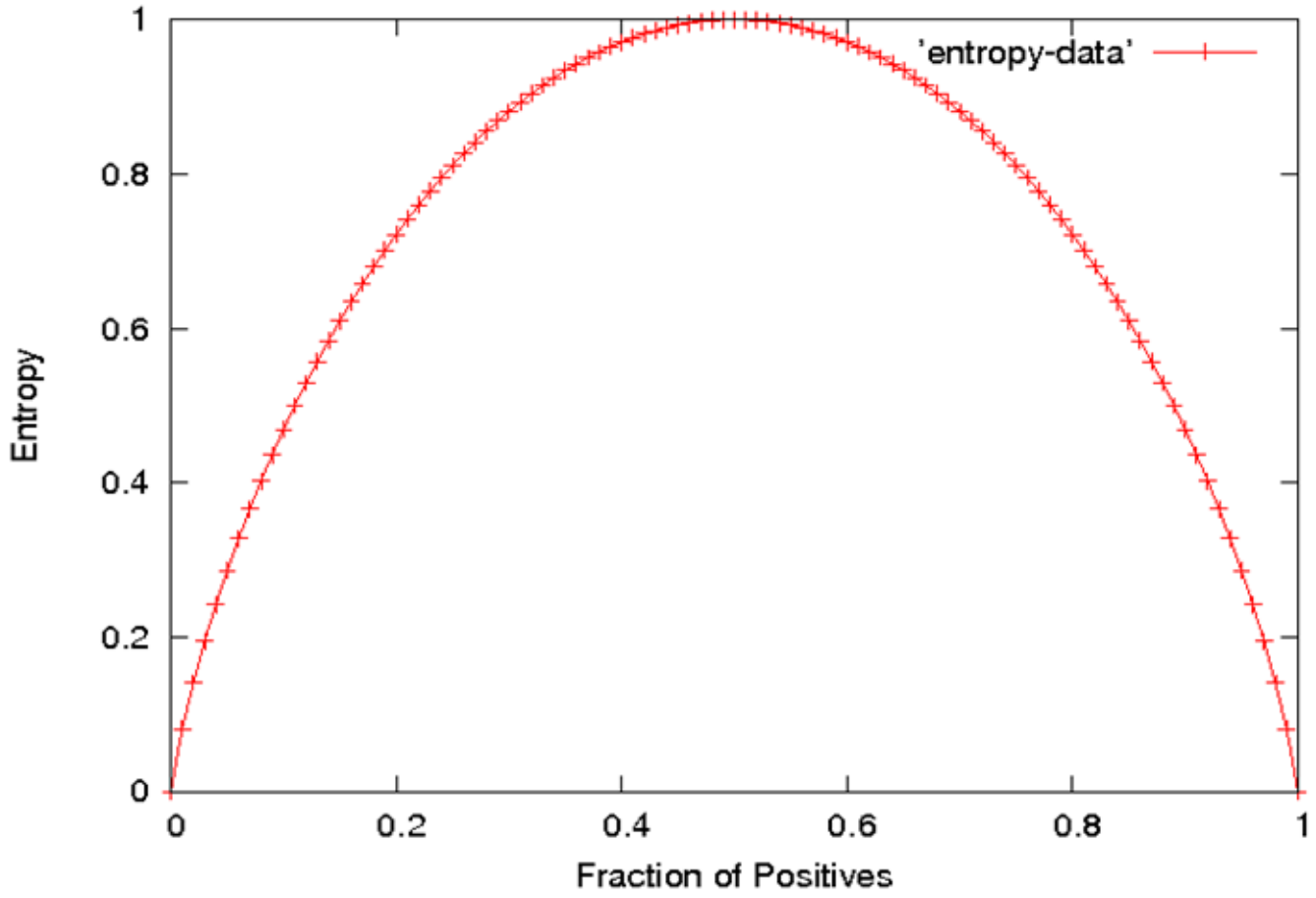
- Entropy (disorder, impurity) of a set of examples, S, relative to a binary classification is:

$$Entropy(S) = -p_1 \log_2(p_1) - p_0 \log_2(p_0)$$

  where p_1 is the fraction of positive examples in S and p_0 is the fraction of negatives.
- If all examples are in one category, entropy is zero (we define $0 \cdot \log(0)=0$)
- If examples are equally mixed ($p_1=p_0=0.5$), entropy is a maximum of 1.
- Entropy can be viewed as the number of bits required on average to encode the class of an example in *S* where data compression (e.g. Huffman coding) is used to give shorter codes to more likely cases.
- For multi-class problems with c categories, entropy generalizes to:

$$Entropy(S) = \sum_{i=1}^{c} -p_i \log_2(p_i)$$

Entropy Plot for Binary Classification



Information Gain

- The information gain of a feature *F* is the expected reduction in entropy resulting from splitting on this feature / attribute.

$$Gain(S, F) = Entropy(S) - \sum_{v \in Values(F)} \frac{|S_v|}{|S|} Entropy(S_v)$$

where S_v is the subset of *S* having value *v* for feature *F*.

Information Gain

- Example:
  - <big, red, circle>: + <small, red, circle>: +
  - <small, red, square>: − <big, blue, circle>: −

2+, 2 −: E=1

size

big: 1+,1− E=1

small: 1+,1− E=1

Gain=1−(0.5·1 + 0.5·1) = 0

2+, 2 − : E=1

color

red: 2+,1− E=0.918

blue: 0+,1− E=0

Gain=1−(0.75·0.918 + 0.25·0) = 0.311

2+, 2 − : E=1

shape

circle: 2+,1− E=0.918

square: 0+,1− E=0

Gain=1−(0.75·0.918 + 0.25·0) = 0.311

UNIVERSITI MALAYA

Information Gain

Which attribute is the best classifier?

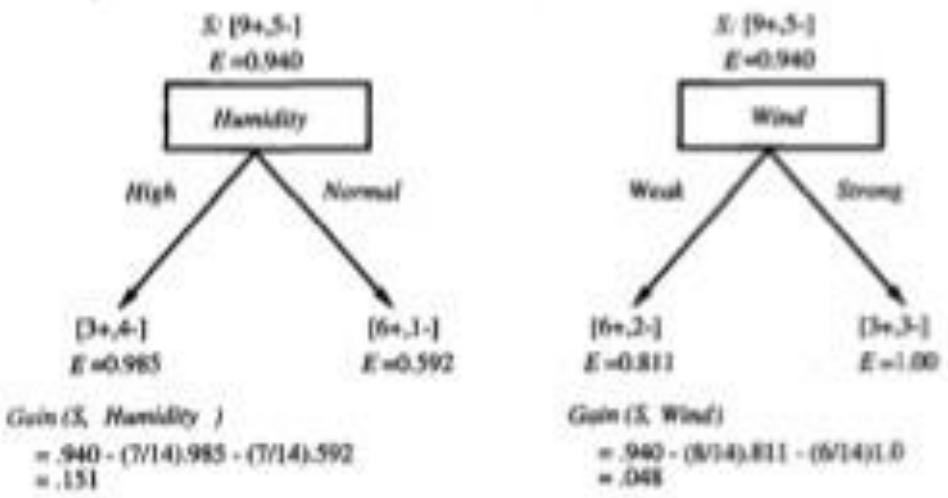


FIGURE 3.3
Humidity provides greater information gain than *Wind*, relative to the target classification. Here, E stands for entropy and S for the original collection of examples. Given an initial collection S of 9 positive and 5 negative examples, [9+, 5−], sorting these by their *Humidity* produces collections of [3+, 4−] ($Humidity = High$) and [6+, 1−] ($Humidity = Normal$). The information gained by this partitioning is .151, compared to a gain of only .048 for the attribute *Wind*.

Hypothesis Space Search

- Performs ***batch learning*** that processes all training instances at once rather than ***incremental learning*** that updates a hypothesis after each example.
- Performs hill-climbing (greedy search) that may only find a locally-optimal solution. Guaranteed to find a tree consistent with any conflict-free training set (i.e. identical feature vectors always assigned the same class), but not necessarily the simplest tree.
- Finds a single discrete hypothesis, so there is no way to provide confidences or create useful queries.

Bias in Decision-Tree Induction

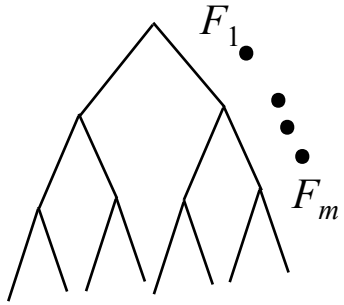
- Information-gain gives a bias for trees with minimal depth.
- Implements a search (preference) bias instead of a language (restriction) bias.

History of Decision-Tree Research

- Hunt and colleagues use exhaustive search decision-tree methods (CLS) to model human concept learning in the 1960's.
- In the late 70's, Quinlan developed ID3 with the information gain heuristic to learn expert systems from examples.
- Simultaneously, Breiman and Friedman and colleagues develop CART (Classification and Regression Trees), similar to ID3.
- In the 1980's a variety of improvements are introduced to handle noise, continuous features, missing features, and improved splitting criteria. Various expert-system development tools results.
- Quinlan's updated decision-tree package (C4.5) released in 1993.
- Weka includes Java version of C4.5 called J48.

Computational Complexity

- Worst case builds a complete tree where every path test every feature. Assume *n* examples and *m* features.



Maximum of *n* examples spread across all nodes at each of the *m* levels

- At each level, *i*, in the tree, must examine the remaining *m*− *i* features for each instance at the level to calculate info gains.

$$\sum_{i=1}^{m} i \cdot n = O(nm^2)$$

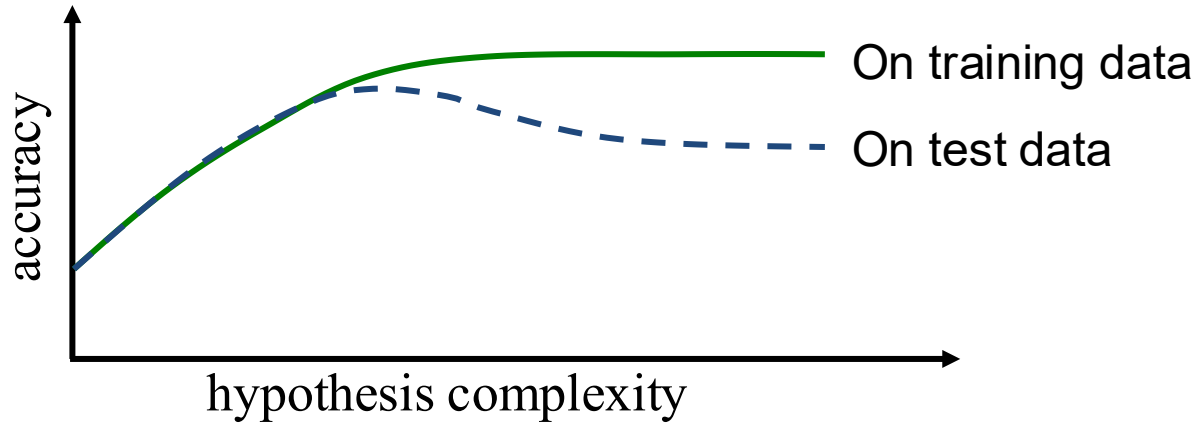
- However, learned tree is rarely complete (number of leaves is ≤ *n*). In practice, complexity is linear in both number of features (*m*) and number of training examples (*n*).

Overfitting

- Learning a tree that classifies the training data perfectly may not lead to the tree with the best generalization to unseen data.
  - There may be noise in the training data that the tree is erroneously fitting.
  - The algorithm may be making poor decisions towards the leaves of the tree that are based on very little data and may not reflect reliable trends.

Overfitting

- A hypothesis, *h*, is said to over fit the training data is there exists another hypothesis which, *h´*, such that *h* has less error than *h´* on the training data but greater error on independent test data.

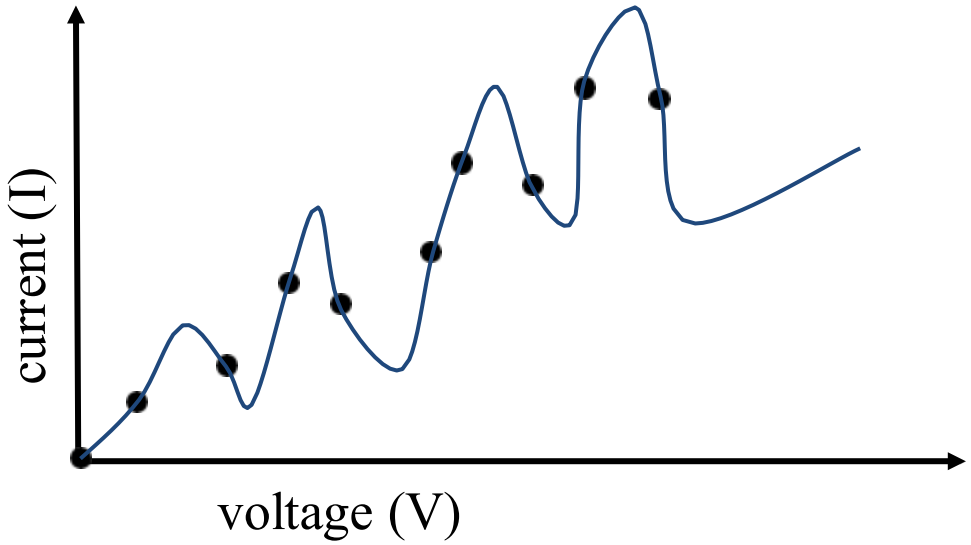


Overfitting

Testing Ohms Law: V = IR (I = (1/R)V)

Experimentally measure 10 points

Fit a curve to the Resulting data.

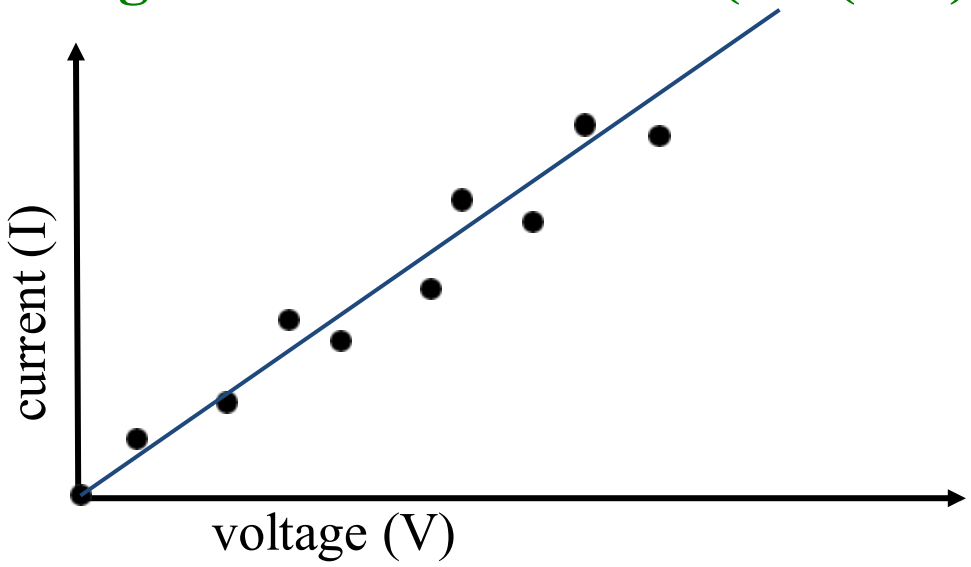


Perfect fit to training data with an 9th degree polynomial
(can fit *n* points exactly with an *n*-1 degree polynomial)

Ohm was wrong, we have found a more accurate function!

Overfitting

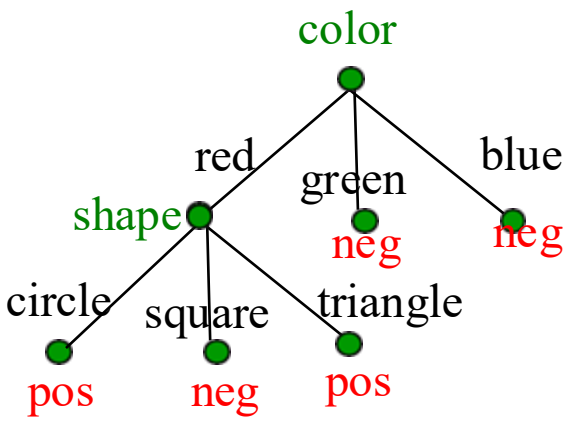
Testing Ohms Law: V = IR (I = (1/R)V)

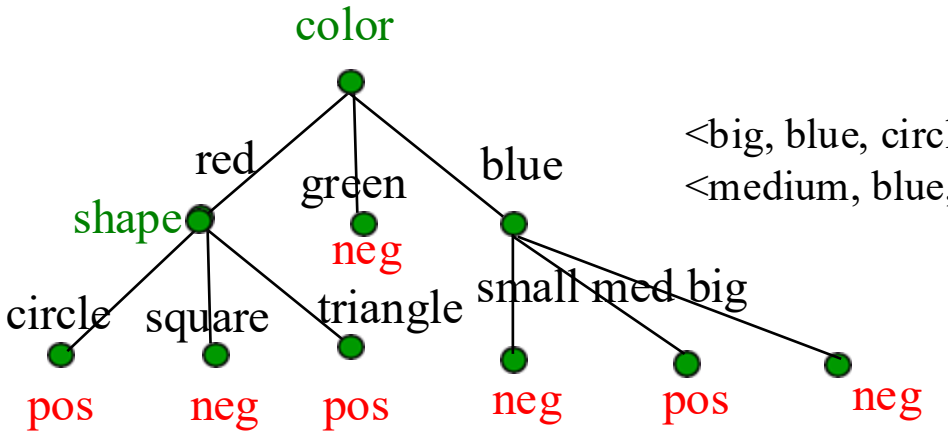


Better generalization with a linear function that fits training data less accurately.

Overfitting Noise in Decision Trees

- Category or feature noise can easily cause overfitting.
  - Add noisy instance <medium, blue, circle>: pos (but really neg)





<big, blue, circle>: –
<medium, blue, circle>: +

Overfitting Noise in Decision Trees

- Noise can also cause different instances of the same feature vector to have different classes. Impossible to fit this data and must label leaf with the majority class.
  - <big, red, circle>: neg (but really pos)
- Conflicting examples can also arise if the features are incomplete and inadequate to determine the class or if the target concept is non-deterministic.

Overfitting Prevention (Pruning) Methods

- Two basic approaches for decision trees
  - Prepruning: Stop growing tree as some point during top-down construction when there is no longer sufficient data to make reliable decisions.
  - Postpruning: Grow the full tree, then remove subtrees that do not have sufficient evidence.
- Label leaf resulting from pruning with the majority class of the remaining data, or a class probability distribution.
- Method for determining which subtrees to prune:
  - Cross-validation: Reserve some training data as a hold-out set (***validation set***, ***tuning set***) to evaluate utility of subtrees.
  - Statistical test: Use a statistical test on the training data to determine if any observed regularity can be dismisses as likely due to random chance.
  - Minimum description length (MDL): Determine if the additional complexity of the hypothesis is less complex than just explicitly remembering any exceptions resulting from pruning.

Reduced Error Pruning

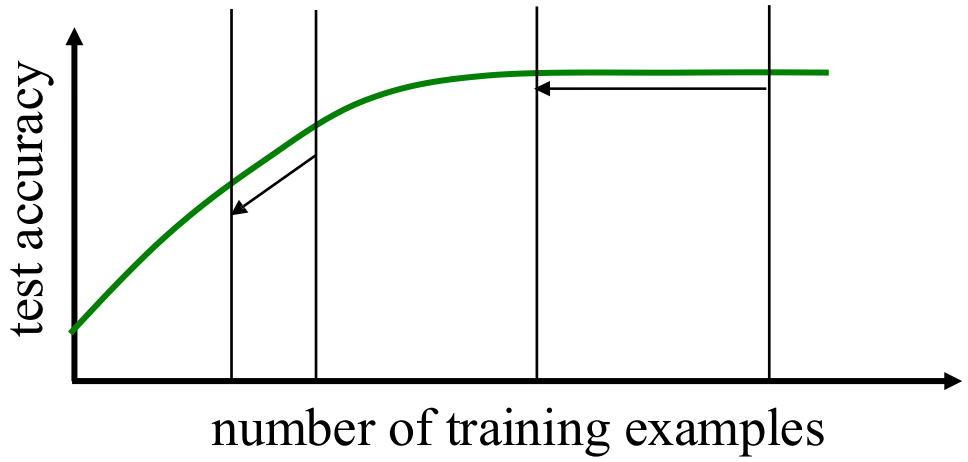
- A post-pruning, cross-validation approach.

Partition training data in "grow" and "validation" sets.
Build a complete tree from the "grow" data.
Until accuracy on validation set decreases do: For each
non-leaf node, *n*, in the tree do:
Temporarily prune the subtree below *n* and replace it with a leaf
labeled with the current majority class at that node.
Measure and record the accuracy of the pruned tree on the validation set.

Permanently prune the node that results in the greatest increase in accuracy on the
validation set.

Issues with Reduced Error Pruning

- The problem with this approach is that it potentially "wastes" training data on the validation set.
- Severity of this problem depends where we are on the learning curve:



Cross-Validating without Losing Training Data

- If the algorithm is modified to grow trees breadth-first rather than depth-first, we can stop growing after reaching any specified tree complexity.
- First, run several trials of reduced error-pruning usingdifferent random splits of grow and validation sets.
- Record the complexity of the pruned tree learned in each trial. Let *C* be the average pruned-tree complexity.
- Grow a final tree breadth-first from all the training data but stop when the complexity reaches *C*.
- Similar cross-validation approach can be used to set arbitrary algorithm parameters in general.

Additional Decision Tree Issues

- Better splitting criteria
  - Information gain prefers features with many values.
- Continuous features
- Predicting a real-valued function (regression trees)
- Missing feature values
- Features with costs
- Misclassification costs
- Incremental learning
  - ID4
  - ID5
- Mining large databases that do not fit in main memory

What is ID3?

- A mathematical algorithm for building the decision tree.
- Invented by J. Ross Quinlan in 1979.
- Uses Information Theory invented by Shannon in 1948.
- Builds the tree from the top down, with no backtracking.
- Information Gain is used to select the most useful attribute for classification.

Information Gain (IG)

- The information gain is based on the decrease in entropy after a dataset is split on an attribute.
- Which attribute creates the most homogeneous branches?
- First the entropy of the total dataset is calculated.
- The dataset is then split on the different attributes.
- The entropy for each branch is calculated. Then it is added proportionally, to get total entropy for the split.
- The resulting entropy is subtracted from the entropy before the split.
- The result is the Information Gain, or decrease in entropy.
- The attribute that yields the largest IG is chosen for the decision node.

Information Gain (cont'd)

- A branch set with entropy of 0 is a leaf node.
- Otherwise, the branch needs further splitting to classify its dataset.
- The ID3 algorithm is run recursively on the non-leaf branches, until all data is classified.

Advantages of using ID3

- Understandable prediction rules are created from the training data.
- Builds the fastest tree.
- Builds a short tree.
- Only need to test enough attributes until all data is classified.
- Finding leaf nodes enables test data to be pruned, reducing number of tests.
- Whole dataset is searched to create tree.

Disadvantages of using ID3

- Data may be over-fitted or over-classified, if a small sample is tested.
- Only one attribute at a time is tested for making a decision.
- Classifying continuous data may be computationally expensive, as many trees must be generated to see where to break the continuum.

Example: The Simpsons

Person	Hair Length	Weight	Age	Class
Homer	0″	250	36	**M**
Marge	10″	150	34	**F**
Bart	2″	90	10	**M**
Lisa	6″	78	8	**F**
Maggie	4″	20	1	**F**
Abe	1″	170	70	**M**
Selma	8″	160	41	**F**
Otto	10″	180	38	**M**
Krusty	6″	200	45	**M**

Comic	8″	290	38	**?**

$$Entropy(S) = -\frac{p}{p+n}\log_2\left(\frac{p}{p+n}\right) - \frac{n}{p+n}\log_2\left(\frac{n}{p+n}\right)$$

$$Entropy(4\mathbf{F},5\mathbf{M}) = -(4/9)\log_2(4/9) - (5/9)\log_2(5/9)$$
$$= \mathbf{0.9911}$$

yes Hair Length <= 5? no

Let us try splitting on *Hair length*

$$Entropy(1\mathbf{F},3\mathbf{M}) = -(1/4)\log_2(1/4) - (3/4)\log_2(3/4)$$
$$= \mathbf{0.8113}$$

$$Entropy(3\mathbf{F},2\mathbf{M}) = -(3/5)\log_2(3/5) - (2/5)\log_2(2/5)$$
$$= \mathbf{0.9710}$$

$$Gain(A) = E(Current\ set) - \sum E(all\ child\ sets)$$

$$Gain(\text{Hair Length} <= 5) = \mathbf{0.9911} - (4/9 * \mathbf{0.8113} + 5/9 * \mathbf{0.9710}) = \mathbf{0.0911}$$

$$Entropy(S) = -\frac{p}{p+n}\log_2\left(\frac{p}{p+n}\right) - \frac{n}{p+n}\log_2\left(\frac{n}{p+n}\right)$$

$$Entropy(4\mathbf{F},5\mathbf{M}) = -(4/9)\log_2(4/9) - (5/9)\log_2(5/9)$$
$$= \mathbf{0.9911}$$

yes

Weight <= 160?

no

Let us try splitting on *Weight*

$$Entropy(4\mathbf{F},1\mathbf{M}) = -(4/5)\log_2(4/5) - (1/5)\log_2(1/5)$$
$$= \mathbf{0.7219}$$

$$Entropy(0\mathbf{F},4\mathbf{M}) = -(0/4)\log_2(0/4) - (4/4)\log_2(4/4)$$
$$= \mathbf{0}$$

$$Gain(A) = E(Current\ set) - \sum E(all\ child\ sets)$$

$$Gain(\text{Weight} <= 160) = \mathbf{0.9911} - (5/9 * \mathbf{0.7219} + 4/9 * \mathbf{0}) = \mathbf{0.5900}$$

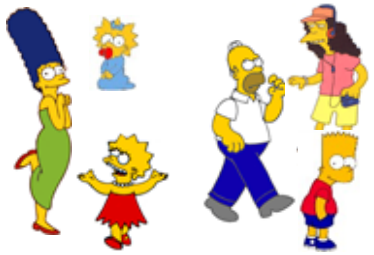
$$Entropy(S) = -\frac{p}{p+n}\log_2\left(\frac{p}{p+n}\right) - \frac{n}{p+n}\log_2\left(\frac{n}{p+n}\right)$$

$$Entropy(4\mathbf{F},5\mathbf{M}) = -(4/9)\log_2(4/9) - (5/9)\log_2(5/9)$$
$$= \mathbf{0.9911}$$

yes

age <= 40?

no



Let us try splitting on *Age*

$$Entropy(3\mathbf{F},3\mathbf{M}) = -(3/6)\log_2(3/6) - (3/6)\log_2(3/6)$$
$$= \mathbf{1}$$

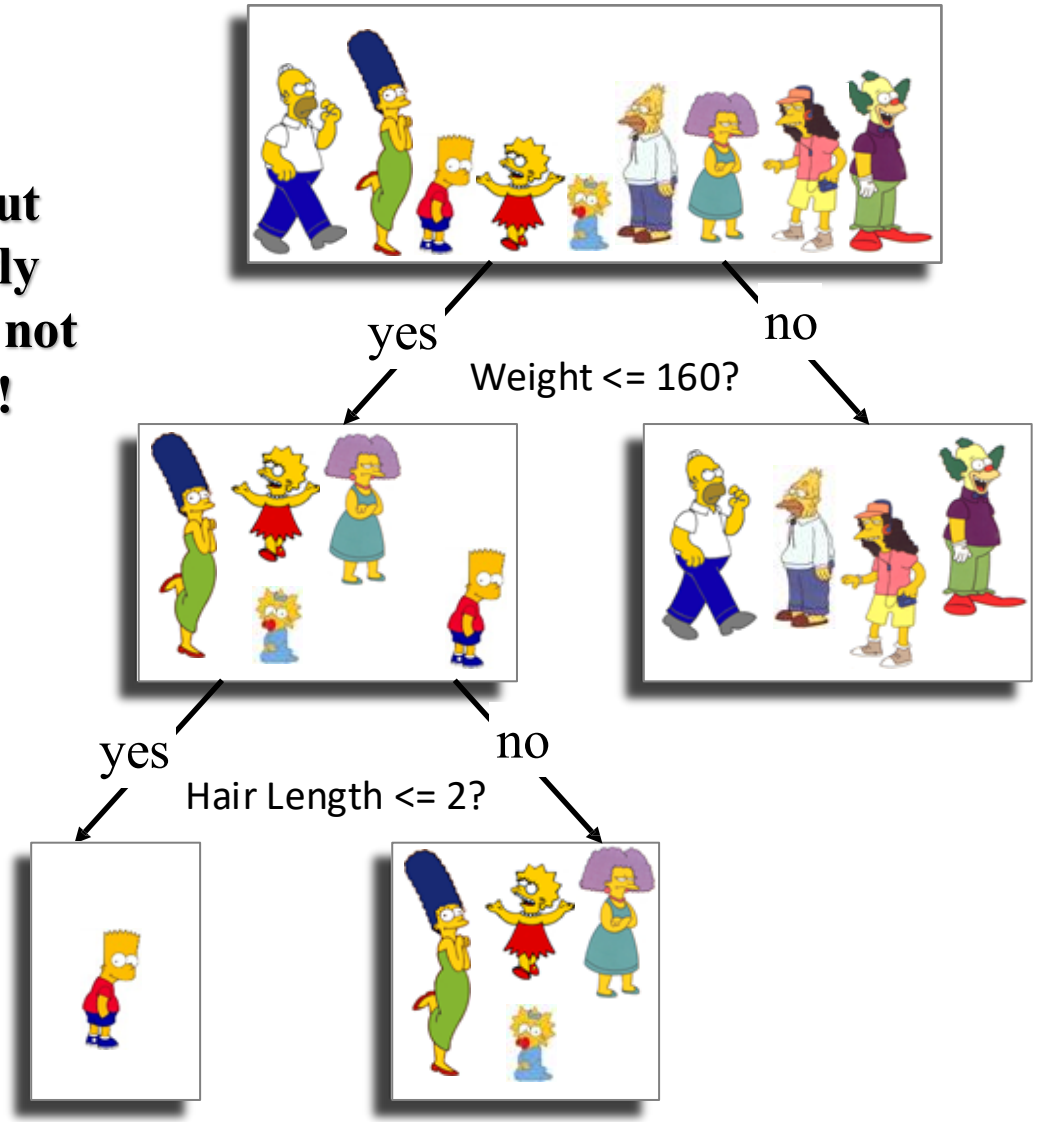
$$Entropy(1\mathbf{F},2\mathbf{M}) = -(1/3)\log_2(1/3) - (2/3)\log_2(2/3)$$
$$= \mathbf{0.9183}$$

$$Gain(A) = E(Current\ set) - \sum E(all\ child\ sets)$$

$$Gain(\text{Age} <= 40) = \mathbf{0.9911} - (6/9 * \mathbf{1} + 3/9 * \mathbf{0.9183}) = \mathbf{0.0183}$$

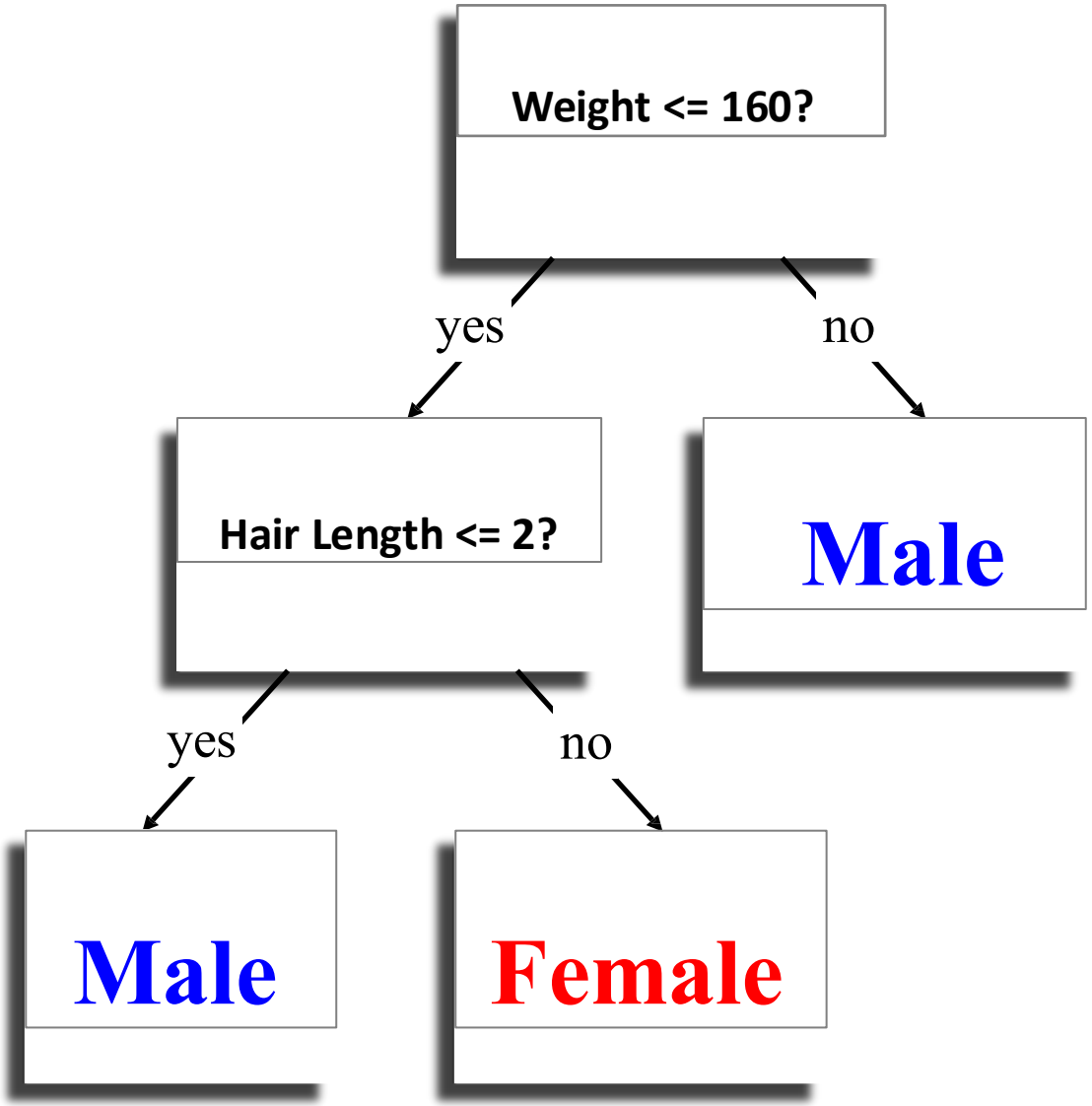
Of the 3 features we had, *Weight* was best. But while people who weigh over 160 are perfectly classified (as males), the under 160 people are not perfectly classified… So, we simply recurse!

This time we find that we can split on *Hair length,* and we are done!

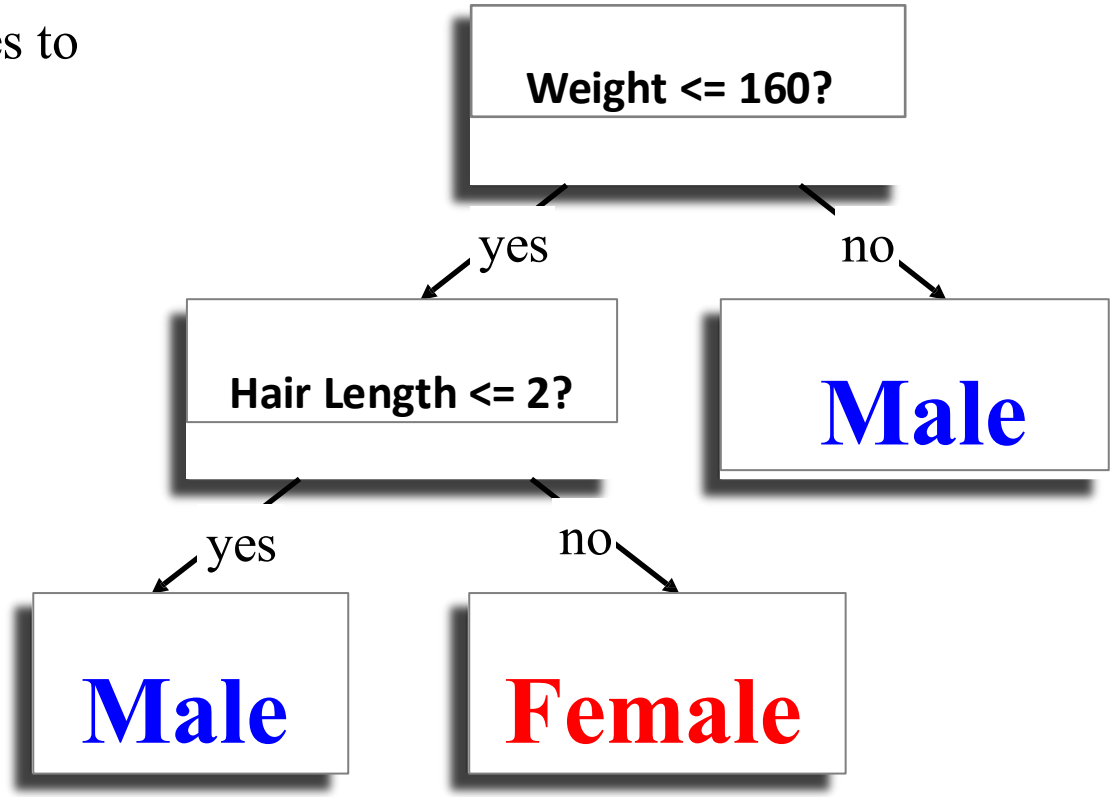


We need don't need to keep the data around, just the test conditions.

How would these people be classified?



It is trivial to convert Decision Trees to rules…



Rules to Classify Males/Females

If *Weight* **greater than** 160, classify as **Male**
Elseif *Hair Length* **less than or equal** to 2, classify as **Male**
Else classify as **Female**

SUMMARY AND FURTHER READING

The main points of this chapter include:

- Decision tree learning provides a practical method for concept learning and for learning other discrete-valued functions. The ID3 family of algorithms infers decision trees by growing them from the root downward, greedily selecting the next best attribute for each new decision branch added to the tree.
- ID3 searches a complete hypothesis space (i.e., the space of decision trees can represent any discrete-valued function defined over discrete-valued instances). It thereby avoids the major difficulty associated with approaches that consider only restricted sets of hypotheses: that the target function might not be present in the hypothesis space.
- The inductive bias implicit in ID3 includes a preference for smaller trees; that is, its search through the hypothesis space grows the tree only as large as needed in order to classify the available training examples.

SUMMARY AND FURTHER READING

- Overfitting the training data is an important issue in decision tree learning. Because the training examples are only a sample of all possible instances, it is possible to add branches to the tree that improve performance on the training examples while decreasing performance on other instances outside this set. Methods for post-pruning the decision tree are therefore important to avoid overfitting in decision tree learning (and other inductive inference methods that employ a preference bias).
- A large variety of extensions to the basic ID3 algorithm has been developed by different researchers. These include methods for post-pruning trees, handling real- valued attributes, accommodating training examples with missing attribute values, incrementally refining decision trees as new training examples become available, using attribute selection measures other than information gain, and considering costs associated with instance attributes.

References

- Quinlan, J.R. 1986, Machine Learning, 1, 81
- http://dms.irb.hr/tutorial/tut_dtrees.php
- http://www.dcs.napier.ac.uk/~peter/vldb/dm/node11.html
- http://www2.cs.uregina.ca/~dbd/cs831/notes/ml/dtrees/4_dtrees2.html
- Professor Sin-Min Lee, SJSU. http://cs.sjsu.edu/~lee/cs157b/cs157b.html

THANK YOU